==========
Pre-dependent haskell

Usage: pattern matching by type
Steps
- use **withSing** to reflect the type i.e. reflect from type => singleton
- **pattern match** on singleton

e.g. dependent type annotated with Encoding

```
{-# LANGUAGE GADTs #-}
{-# LANGUAGE DataKinds #-}
{-# LANGUAGE StandaloneKindSignatures #-}
{-# LANGUAGE TemplateHaskell #-}
{-# LANGUAGE TypeFamilies #-}
{-# LANGUAGE LambdaCase #-}

import Data.ByteString qualified as B
import Data.Kind (Type)
import Data.Singletons
import Data.Singletons.TH
import Data.Text qualified as T
import Data.Text.Encoding

data FileEncoding = Latin1 | UTF8 | UTF16 | UTF32

genSingletons [''FileEncoding]

type DataFile :: FileEncoding -> Type
data DataFile a where
  DataFile :: SingI a => FilePath -> DataFile a

readDataFile :: forall a. DataFile a -> IO T.Text
readDataFile (DataFile path) = withSing @a $ \case
        -- withSing: reflect from type to value i.e. a
-> Sing a
    SLatin1 -> decodeLatin1 <$> B.readFile path
        -- pattern match
    SUTF8 -> decodeUtf8 <$> B.readFile path
    SUTF16 -> decodeUtf16BE <$> B.readFile path
    SUTF32 -> decodeUtf32BE <$> B.readFile path
```

GOOD: pattern match

```
33 readDataFile :: forall a. DataFile a -> IO T.Text
34 readDataFile (DataFile path) = withSing @a $ \case
35     SLatin1 -> decodeLatin1 <$> B.readFile path
36     SUTF8 -> decodeUtf8 <$> B.readFile path
37     SUTF16 -> decodeUtf16BE <$> B.readFile path
38     SUTF32 -> decodeUtf32BE <$> B.readFile path
```

BAD: incomplete pattern match

```
33 readDataFile :: forall a. DataFile a -> IO T.Text
34 readDataFile (DataFile path) = withSing @a $ \case      ■ Pattern m
35     -- SLatin1 -> decodeLatin1 <$> B.readFile path
36     SUTF8 -> decodeUtf8 <$> B.readFile path
37     SUTF16 -> decodeUtf16BE <$> B.readFile path
38     SUTF32 -> decodeUtf32BE <$> B.readFile path
39
```

GOTCHA: naive **reification** i.e. instantiating a **dependent type from a plain value**
Problem:

mkMyData1 :: forall (a ::Bool). Bool -> MyData a BAD. MUST NOT reify from term to type directly.
mkMyData2 :: forall (a ::Bool). Sing a-> MyData a OK. With **witness** (singleton)
mkMyData2b :: forall (a ::Bool). Proxy a-> MyData a OK. With proxy **witness**
mkMyData3 :: Bool -> SomeMyData OK. With **opaque container**

mkMyData1 is bad since

mkMyData1 True :: MyData 'True MyData 'True
mkMyData1 True :: MyData 'False BAD. Expected MyData 'True but can force it to MyData 'False

e.g.

```
{-# LANGUAGE DataKinds #-}
{-# LANGUAGE FlexibleContexts #-}
{-# LANGUAGE FlexibleInstances #-}
{-# LANGUAGE GADTs #-}
{-# LANGUAGE InstanceSigs #-}
{-# LANGUAGE OverloadedStrings #-}
{-# LANGUAGE PolyKinds #-}
{-# LANGUAGE RankNTypes #-}
{-# LANGUAGE StandaloneKindSignatures #-}
{-# LANGUAGE TemplateHaskell #-}
{-# LANGUAGE TypeFamilies #-}

import Data.Kind
import Data.Singletons.TH

-- setup: generate Bool singletons
genSingletons [''Bool]

-- setup: main data type
type MyData :: Bool -> Type
data MyData a where
  MkMyData :: MyData a
```

```
-- solution 1: fix type with witness
mkMyData1 :: Sing (a :: Bool) -> MyData a
     -- singleton Sing (a :: Bool) to fix type
mkMyData1 v = withSingI v MkMyData


-- solution 2: with opaque container
data SomeData where
       -- opaque container SomeData
  MkSomeData :: forall (a :: Bool). MyData a ->
SomeData
mkMyData :: Bool -> SomeData
mkMyData v = withSomeSing v $
  \(s :: SBool d) -> withSingI s $
    MkSomeData (MkMyData @d)
```

GOTCHA: output type variable is bad

Problem:

Similar to the reification GOTCHA above.

f1 :: forall (a :: Bool), (b :: Bool) => MyData a -> MyData b BAD
f2 :: forall (a :: Bool). MyData a -> MyData (F a) OK. Parameterize with type family
f3 :: forall (a :: Bool). MyData a -> SomeMyData OK. With opaque container

Again, f1 is bad since you can enforce any value
e.g
f1 x :: MyData 'True -- OK
f1 x :: MyData 'False -- OK even though same input

e.g.

```
{-# LANGUAGE DataKinds #-}
{-# LANGUAGE GADTs #-}
{-# LANGUAGE PolyKinds #-}
{-# LANGUAGE StandaloneKindSignatures #-}
{-# LANGUAGE TemplateHaskell #-}
{-# LANGUAGE TypeFamilies #-}

import Data.Singletons.TH

genSingletons [''Bool]

-- problem: bad output type
notBad :: forall (a :: Bool) (b :: Bool). SingI b =>
Sing a -> Sing b
notBad STrue = SFalse
-- ERROR. "Couldn't deduce b ~ False"
notBad _ = sing
  -- BAD.Compiles but doesn't work as expected


-- solution 1: compute with type family
```

```
type Not :: Bool -> Bool
type family Not a where
  Not 'True = 'False
  Not 'False = 'True
sNot1 :: forall (a :: Bool). Sing a -> Sing (Not a)
sNot1 STrue = SFalse
sNot1 SFalse = STrue


-- solution 2: opaque container
sNot2 :: forall (a :: Bool). Sing a -> SomeSing Bool
sNot2 STrue = SomeSing SFalse
sNot2 SFalse = SomeSing STrue


-- e.g.
notBad STrue :: SBool 'False
notBad STrue :: SBool 'True        -- BAD. Different
output with same input
sNot1 STrue :: SBool 'False          -- OK
sNot1 SFalse :: SBool 'True          -- OK
sNot1 STrue :: SBool 'True           -- GOOD ERROR as
expected "Couldn't match type 'False with 'True"
sNot2 STrue :: SomeSing Bool        -- OK opaque
sNot2 SFalse :: SomeSing Bool       -- OK opage
```

Usage: **type-parameter pattern matching** with type families and classes
2 main techniques

type family	type parameter pattern match to compute type
class	type parameter pattern match to compute value

e.g.

```
{-# LANGUAGE DataKinds #-}
{-# LANGUAGE PolyKinds #-}
{-# LANGUAGE StandaloneKindSignatures #-}
{-# LANGUAGE TypeOperators #-}
{-# LANGUAGE TypeFamilies #-}
{-# LANGUAGE UndecidableInstances #-}

import Data.Kind
import Data.Proxy
import GHC.TypeLits

-- setup: base types
data Value = A | B | C | D
type Fn1 :: forall a. a -> Type
data Fn1 a
type Fn2 :: forall a. a -> Type
data Fn2 a
type (:+) :: forall a b. a -> b -> Type
data a :+ b
```

```
-- method 1: pattern match + map type -> type (Symbol)
type ToSymbol :: forall a. a -> Symbol
type family ToSymbol a where
  ToSymbol 'A = "A"
  ToSymbol 'B = "B"
  ToSymbol 'C = "C"
  ToSymbol 'D = "D"
  ToSymbol (a :+ b) = AppendSymbol (ToSymbol a)
(AppendSymbol " " (ToSymbol b))
  ToSymbol (Fn1 a) = AppendSymbol (AppendSymbol "Fn1("
(ToSymbol a)) ")"
  ToSymbol (Fn2 a) = AppendSymbol (AppendSymbol "Fn2("
(ToSymbol a)) ")"


-- method 2: pattern match + map type -> values (Int)
type HasSecretCode :: forall a. a -> Constraint
class HasSecretCode a where
  getSecreteCode :: Proxy a -> Int
instance HasSecretCode 'A where getSecreteCode _ = 1
instance HasSecretCode 'B where getSecreteCode _ = 2
instance HasSecretCode 'C where getSecreteCode _ = 3
instance HasSecretCode 'D where getSecreteCode _ = 4
instance HasSecretCode a => HasSecretCode (Fn1 a) where
getSecreteCode _ = getSecreteCode (Proxy @a) * 10
instance HasSecretCode a => HasSecretCode (Fn2 a) where
getSecreteCode _ = negate $ getSecreteCode (Proxy @a)
instance (HasSecretCode a, HasSecretCode b) =>
HasSecretCode (a :+ b) where getSecreteCode _ =
getSecreteCode (Proxy @a) + getSecreteCode (Proxy @b)


-- e.g. information as type
type V1 = 'A :+ 'B
type V2 = 'A :+ 'B :+ 'C :+ 'D
type V3 = (Fn1 (Fn2 (Fn1 'A)) :+ Fn1 V1) :+ Fn2 V1

-- map to type -> type
symbolVal (Proxy @(ToSymbol V1))        -- ToSymbol V1 =
"A B" :: Symbol
symbolVal (Proxy @(ToSymbol V2))        -- "A B C D"
symbolVal (Proxy @(ToSymbol V3))        --
"Fn1(Fn2(Fn1(A))) Fn1(A B) Fn2(A B)"

-- map to type -> value
getSecreteCode (Proxy @V1)            -- 3 :: Int
getSecreteCode (Proxy @V2)            -- 10 :: Int
getSecreteCode (Proxy @V3)            -- -73 :: Int
```

Usage: use content of **opaque type**
Use **type signature** in pattern

e.g. with SomeSymbol

SomeSymbol (_ :: Proxy s)	content s :: Symbol

```
{-# LANGUAGE DataKinds #-}
{-# LANGUAGE PolyKinds #-}
{-# LANGUAGE StandaloneKindSignatures #-}

import Data.Kind
import GHC.TypeLits

type MyType :: Symbol -> Type
data MyType (s :: Symbol) where
  MyType :: KnownSymbol s => MyType s

data SomeMyType = forall s. SomeMyType (MyType s)
              -- custom opaque type

-- e.g. convert between opaque types
toMyType :: SomeSymbol -> SomeMyType
toMyType (SomeSymbol (_ :: Proxy s)) = SomeMyType $
MyType @s

toSomeSymbol :: SomeMyType -> SomeSymbol
toSomeSymbol (SomeMyType (MyType :: MyType s)) =
SomeSymbol $ Proxy @s
```

Example**:** natural numbers as a type parameter

In addition to defining natural numbers, supports **basic operations** i.e. (+), (-), (<=)

data Nat = Z | S Nat	definition
f (m :: Nat)	as a type parameter
f (m + n :: Nat)	can add
f (m - n :: Nat)	can sub
(m <= n ~ 'True) => ...	can compare

Base definition

```
{-# LANGUAGE AllowAmbiguousTypes #-}
{-# LANGUAGE DataKinds #-}
{-# LANGUAGE EmptyCase #-}
{-# LANGUAGE FlexibleContexts #-}
{-# LANGUAGE GADTs #-}
{-# LANGUAGE ImpredicativeTypes #-}
{-# LANGUAGE InstanceSigs #-}
{-# LANGUAGE LambdaCase #-}
{-# LANGUAGE PatternSynonyms #-}
{-# LANGUAGE PolyKinds #-}
{-# LANGUAGE StandaloneKindSignatures #-}
{-# LANGUAGE TemplateHaskell #-}
{-# LANGUAGE TypeFamilies #-}
```

```
{-# LANGUAGE TypeOperators #-}
{-# LANGUAGE ViewPatterns #-}
{-# OPTIONS_GHC -Werror=incomplete-patterns #-}

import Control.Monad
import Data.Foldable
import Data.Function ((&))
import Data.Singletons.Decide
import Data.Singletons.TH
import Data.Type.Equality hiding (type (==))
import GHC.Natural
import Prelude hiding (pred, succ)

data Nat = Z | S Nat deriving (Eq, Show)

genSingletons [''Nat]

instance Eq (SNat n) where _ == _ = True

instance Show (SNat n) where show = showSNat

instance SDecide Nat where
  (%~) :: SNat n -> SNat m -> Decision (n :~: m)
  SZ %~ SZ = Proved Refl
  (SS m) %~ (SS n) = case m %~ n of
    Proved Refl -> Proved Refl
    Disproved f -> Disproved $ \Refl -> f Refl
  (SS _) %~ SZ = Disproved $ \case {}
  SZ %~ (SS _) = Disproved $ \case {}

toNatural :: SNat n -> Natural
toNatural SZ = 0
toNatural (SS x) = 1 + toNatural x

showSNat :: SNat n -> String
showSNat x = "SNat @" <> show (toNatural x)

suc :: SNat n -> SNat ('S n)
suc SZ = SS SZ
suc n@(SS _) = SS n

pred :: (n ~ 'S m) => SNat n -> SNat m
pred (SS n) = n
```

Opaque type
e.g. for creating a list of numbers

```
type SomeNat = SomeSing Nat

pattern SomeNat :: SNat (n :: Nat) -> SomeSing Nat
pattern SomeNat n = SomeSing n

{-# COMPLETE SomeNat #-}
```

```
showSomeSNat :: SomeNat -> String
showSomeSNat (SomeNat x) = showSNat x

allNats :: [SomeNat]
                -- list of all natural numbers
allNats = iterate (\(SomeNat n) -> SomeNat (SS n))
(SomeNat SZ)

someNat :: Natural -> SomeNat
someNat n = allNats !! fromIntegral n
```

Arithmetic operations (+) and (-) with properties

0 + m = m
m + n = n + m **commutative**
(m + n) + o = m + (n + o) **associative**

m - 0 = m

Definitions

```
type (+) :: Nat -> Nat -> Nat
type family (+) n m where
  'Z + n = n                                -- (SA)
  ('S n) + m = 'S (n + m)                   -- (SB)

type (-) :: Nat -> Nat -> Nat
type family (-) n m where
  n - 'Z = n
  ('S n) - ('S m) = n - m
```

Proofs
Commutativity and **associativity** can not be inferred (why?) so they need to be proved

```
sumFlipZ :: forall n. SNat n -> n + 'Z :~: n
-- 0 + 0      =     0                     by (SA)
sumFlipZ SZ = Refl
-- n + 0      =     (1 + (n - 1)) + 0     by input
pattern
--            =     1 + ((n - 1) + 0)     by (SB)
--            =     1 + (n - 1)           by recursion
--            =     n
sumFlipZ (SS n_) = gcastWith (sumFlipZ n_) Refl

sumFlipS :: forall m n. SNat n -> SNat m -> n + 'S m :~:
'S n + m
-- 0 + (1 + m)      =     1 + m                       by
(SA)
--                  =     1 + (0 + m)                 by
(SA)
--                  =     (1 + 0) + m                 by
```

```
(SB)
sumFlipS SZ _ = Refl
-- n + (1 + m)      =     (1 + (n - 1)) + (1 + m)     by
input pattern
--                  =     1 + ((n - 1) + (1 + m))     by
(SB)
--                  =     1 + (n + m)                 by
recursion
--                  =     (1 + n) + m                 by
(SB)
sumFlipS (SS n_) m = gcastWith (sumFlipS n_ m) Refl

sumIsCommutative' :: forall n m. (SingI n, SingI m) =>
(n + m :~: m + n)
sumIsCommutative' = sumIsCommutative (sing @n) (sing @m)

sumIsCommutative :: forall n m. SNat n -> SNat m -> (n +
m :~: m + n)
-- 0 + m    =     m                       by (SA)
--          =     m + 0                   by sumFlipZ
sumIsCommutative SZ m = gcastWith (sumFlipZ m) Refl
-- n + m    =     (1 + (n - 1)) + m       by pattern
match
--          =     1 + ((n - 1) + m)       by (SB)
--          =     1 + (m + (n - 1))       by recursion
--          =     (1 + m) + (n - 1)       by (SB)
--          =     m + n                   by sumFlipS
sumIsCommutative (SS (n_ :: SNat n_)) m =
  gcastWith (sumIsCommutative n_ m :: n_ + m :~: m + n_)
$
    gcastWith (sumFlipS m n_ :: m + n :~: 'S (m + n_))
Refl

sumIsAssociative' :: forall n m o. (SingI n, SingI m,
SingI o) => (n + m) + o :~: n + (m + o)
sumIsAssociative' = sumIsAssociative (sing @n) (sing @m)
(sing @o)

sumIsAssociative :: forall n m o. SNat n -> SNat m ->
SNat o -> ((n + m) + o :~: n + (m + o))
-- (0 + m) + n      =     m + n
by (SA)
--                  =     (m + n)
--                  =     0 + (m + n)
by (SA)
sumIsAssociative SZ _ _ = Refl
-- (n + m) + o      =     ((1 + (n - 1)) + m) + o
by pattern match
--                  =     (1 + ((n - 1) + m) + o
by (SB)
--                  =     1 + (((n - 1) + m) + o)
by (SB)
--                  =     1 + ((n - 1) + (m + o))
by recursion
```

```
--                   =     n + (m + o)
by (SB)
sumIsAssociative (SS (n_ :: SNat n_)) m o = gcastWith
(sumIsAssociative n_ m o) Refl
```

Comparison (<=) with properties

0 <= m			
m <= n	=>	m + 1 <= n + 1	
m <= n, n <= m	=>	m = n	**symmetry**
m <= n, n <= o	=>	m <= o	**transitive**

Definition

```
type (<=) :: Nat -> Nat -> Bool
type family (<=) m n where
  'Z <= n = 'True
  ('S m) <= ('S n) = m <= n
  _ <= _ = 'False

data IsLE m n where
  IsLE :: (m <= n ~ 'True) => SNat m -> SNat n -> IsLE m
n

instance Eq (IsLE m n) where _ == _ = True

instance Show (IsLE m n) where show = showLEProof

withIsLE :: IsLE m n -> ((m <= n ~ 'True) => r) -> r
withIsLE (IsLE _ _) f = f

showLEProof :: forall m n. IsLE m n -> String
showLEProof (IsLE m n) = "LEProof: " <> showSNat m <> "
<= " <> showSNat n

decideLE :: SNat m -> SNat n -> Decision (IsLE m n)
decideLE SZ n = Proved $ IsLE SZ n
decideLE (SS m_) (SS n_) = case decideLE m_ n_ of
  Proved (IsLE _ _) -> Proved $ IsLE (SS m_) (SS n_)
  Disproved f -> Disproved $ \(IsLE _ _) -> f (IsLE m_
n_)
decideLE (SS _) SZ = Disproved $ \case {}
```

Proofs
Step, symmetry, transitivity cannot be inferred by the compiler (why?) so need to be proved

```
isLEZero :: SNat n -> IsLE 'Z n
isLEZero n = IsLE SZ n

isLESucc :: IsLE m n -> IsLE ('S m) ('S n)
isLESucc (IsLE m n) = IsLE (SS m) (SS n)
```

```
isLEPred :: IsLE ('S m) ('S n) -> IsLE m n
isLEPred (IsLE (SS m_) (SS n_)) = IsLE m_ n_

isLEStep :: IsLE m n -> IsLE m ('S n)
isLEStep (IsLE SZ n) = IsLE SZ (SS n)
isLEStep (IsLE m@(SS m_) n@(SS n_)) = withIsLE (isLEStep
(IsLE m_ n_)) $ IsLE m (SS n)

isLEStepL :: IsLE ('S m) n -> IsLE m n
isLEStepL (IsLE (SS SZ) n) = IsLE SZ n
isLEStepL (IsLE (SS (SS m_)) (SS n_)) = isLESucc
(isLEStepL (IsLE (SS m_) n_))

isLETrans :: IsLE m n -> IsLE n o -> IsLE m o
isLETrans (IsLE SZ _) (IsLE _ o) = IsLE SZ o
isLETrans (IsLE (SS m_) (SS n_)) (IsLE _ (SS o_)) =
isLESucc $ isLETrans (IsLE m_ n_) (IsLE n_ o_)

isLEAsym :: IsLE m n -> IsLE n m -> m :~: n
isLEAsym (IsLE SZ _) (IsLE SZ _) = Refl
isLEAsym (IsLE (SS m_) (SS n_)) (IsLE _ _) = gcastWith
(isLEAsym (IsLE m_ n_) (IsLE n_ m_)) Refl
```

Example application with elevator motion

```
 moveUpM :: SNat m -> Elevator 'DoorClosed n -> Elevator 'DoorClosed (m
+ n)                 m + n
 moveUpM' :: SNat m -> Elevator 'DoorClosed n -> Elevator 'DoorClosed (n
+ m)          same commutative
 moveDownM' :: IsLE m n -> Elevator 'DoorClosed n -> Elevator
'DoorClosed (n - m)              use IsLE
```

```
data DoorState = DoorOpen | DoorClosed deriving (Show,
Eq)

genSingletons [''DoorState]

instance Eq (SDoorState s) where _ == _ = True

instance Show (SDoorState s) where show = showDoorState

showDoorState :: SDoorState s -> String
showDoorState SDoorOpen = "DoorOpen"
showDoorState SDoorClosed = "DoorClosed"

data Elevator s n = Elevator (SDoorState s) (SNat n)

instance Eq (Elevator s n) where _ == _ = True

instance Show (Elevator s n) where show = showElevator

showElevator :: Elevator d n -> String
```

```
showElevator (Elevator d n) = "Elevator " <>
showDoorState d <> " " <> showSNat n

elevatorDoorState :: Elevator d n -> SDoorState d
elevatorDoorState (Elevator d _) = d

elevatorFloor :: Elevator s n -> SNat n
elevatorFloor (Elevator _ n) = n
```

Elevator motion

```
moveUpOne :: Elevator 'DoorClosed n -> Elevator
'DoorClosed (Nat1 + n)
moveUpOne (Elevator d n) = Elevator d (SS n)

moveUpM :: SNat m -> Elevator 'DoorClosed n -> Elevator
'DoorClosed (m + n)
moveUpM SZ e = e
moveUpM (SS m_) e = moveUpOne $ moveUpM m_ e

moveDownOne :: Elevator 'DoorClosed (Nat1 + n) ->
Elevator 'DoorClosed n
moveDownOne (Elevator d (SS n_)) = Elevator d n_

moveDownM :: IsLE m n -> Elevator 'DoorClosed n ->
Elevator 'DoorClosed (n - m)
moveDownM (IsLE SZ _) e = e
moveDownM (IsLE (SS m_) (SS n_)) (Elevator d _) =
moveDownM (IsLE m_ n_) (Elevator d n_)
```

Elevator motion commutative

```
moveUpOne' :: Elevator 'DoorClosed n -> Elevator
'DoorClosed (n + Nat1)
moveUpOne' (Elevator d n) =
  Elevator d (SS n)
    & gcastWith (sumIsCommutative n snat1)

moveUpM' :: SNat m -> Elevator 'DoorClosed n -> Elevator
'DoorClosed (n + m)
moveUpM' SZ e@(Elevator _ n) = gcastWith
(sumIsCommutative SZ n) e
moveUpM' (SS m_) e@(Elevator _ n) =
  moveUpOne' (moveUpM' m_ e)
    & gcastWith (sumIsAssociative n m_ snat1)
    & gcastWith (sumIsCommutative m_ snat1)

-- Note: Is it possible to remove SingI n?
moveDownOne' :: forall n. SingI n => Elevator
'DoorClosed (n + Nat1) -> Elevator 'DoorClosed n
moveDownOne' (Elevator d m) =
  Elevator d (pred m)
```

```
    & gcastWith (sumIsCommutative (sing @n) snat1)

moveDownM' :: IsLE m n -> Elevator 'DoorClosed n ->
Elevator 'DoorClosed (n - m)
moveDownM' (IsLE SZ _) e = e
moveDownM' (IsLE (SS m_) (SS n_)) (Elevator d _) =
moveDownM' (IsLE m_ n_) (Elevator d n_)
```